Parallel Computing - Part 1

Peer Review MP1, Amdahl's Law & Multiprocessing

Week 4

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University

Parallel Computing - Part 1

Today's Agenda

- **Peer Review MP1** (60 min, interleaved)
  - Pairs — one aspect at a time; both show briefly, then compare and discuss
- **Break** (10 min)
- **Theory**: Amdahl, Gustafson & Work-Span (50 min)
  - Laws, limits, back-solving implied serial fractions from measurements
- **Break** (10 min)
- **Studio**: Monte Carlo π + Parallel Mandelbrot
  - Quick milestone overview $\rightarrow$ independent work
  - Soft wrap-up $\sim$15:30; session runs until 16:15

Parallel Computing - Part 1

Pre-Lecture Check

Quick poll — raise your hand:

- Who listened to the podcast?

Parallel Computing - Part 1

Pre-Lecture Check

Quick poll — raise your hand:

- Who listened to the podcast?
- Who watched the video?

Parallel Computing - Part 1

Pre-Lecture Check

Quick poll — raise your hand:

- Who listened to the podcast?
- Who watched the video?
- How many CPU cores does your laptop have?

Python:

`import os; print(os.cpu_count())` — returns *logical* cores (may differ from physical; see later slide)

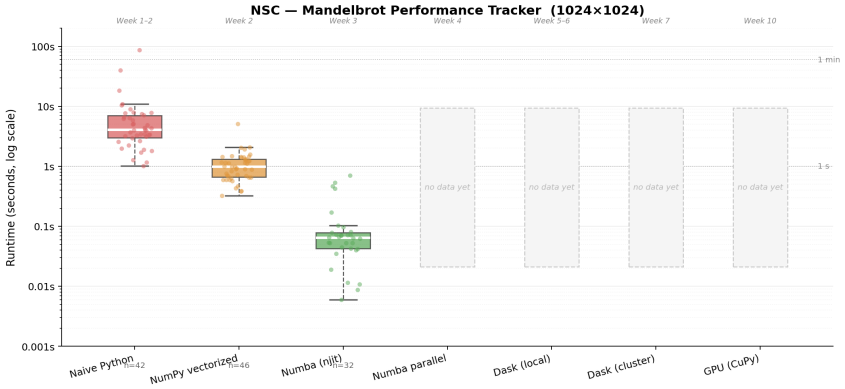
Parallel Computing - Part 1
Learning Objectives

- Learn from a peer's **optimization journey** through MP1
- Understand **Amdahl's Law** and **Gustafson's Law** — and when each applies
- Apply the **Work-Span model** to reason about algorithm parallelism
- Use Python **multiprocessing** to bypass the GIL
- Implement **parallel Mandelbrot** using `Pool.map()`
- Measure **parallel speedup** and efficiency across core counts

By the end of today: peer review done, parallel Mandelbrot running, speedup measured across all core counts.

How Fast Is Your Mandelbrot?

Class Performance Tracker



1024×1024 grid. Boxplot (excl. errors >300 s); dots are individual results.

Updated: March 5, 2026

Physical Peer Review MP1

Peer Review MP1

Instructions

Purpose: Review a fellow student's MP1 work aspect by aspect — compare approaches, discuss what worked.

Format: Interleaved aspect review

Structure:

1. **Setup** (5 min): Pairing and seating; open code and documents
2. **Aspect rounds** (5–10 min each $\times$ 5 aspects): A shows briefly $\rightarrow$ B shows briefly $\rightarrow$ compare and discuss
3. **Reflection** (5 min): Write down two takeaways

Discussion prompts for each aspect:

- What did each person do differently?
- Strengths and weaknesses of each approach?
- What would you keep — and what would you change?

Peer Review MP1

Pair Formation

Pick up a post-it note from the front — colour = your education

- Circulate the room and find someone with a *different* colour
- Sit next to each other and open your code on screen

Goal: pair across educations — different backgrounds ⇒ more to learn from each other

Today's format: aspect-by-aspect review (see next slide)

- Both show their approach to each aspect — then compare and discuss
- No need to present the whole project from start to finish

Remember: Constructive feedback, not criticism!

Peer Review MP1

Aspect-by-Aspect Guide (35 min total)

For each aspect: A shows briefly → B shows briefly → compare and discuss.

1. **Profiling** (∼10 min) — cProfile / line_profiler output; bottleneck identified
   - Where did time go? Did the bottleneck match your expectation?
2. **Numba** (∼10 min) — `@njit` implementation and timings (naive / NumPy / Numba)
   - How did you structure the loop? What speedup did you see?
3. **Data types** (∼5 min) — float32 vs float64 images and timings
   - Which did you choose, and why? What was the accuracy trade-off?
4. **Git history** (∼5 min) — scroll through commit log together
   - Descriptive messages? Does it tell the story of your work?
5. **Code quality** (∼5 min) — read one function each (docstring + body)
   - Is it clear what it does? What would you rename or clarify?

Peer Review MP1

Individual Reflection (5 min)

Write down your answers:

1. What aspect of your partner's implementation surprised you most?
2. What did you learn from your partner's profiling or Numba approach?
3. What will you do differently based on what you saw today?
4. One technique or habit you will carry into MP2?

Peer Review MP1

Class Discussion (5 min)

Share insights with the whole class — volunteers welcome:

- What was the most interesting thing you saw in your partner's work?
- What optimization approach surprised you?
- What creative solution should everyone know about?
- What is one thing you will do differently?

Key lessons:

- Multiple valid approaches to the same problem
- Different hardware $\rightarrow$ different optimal solutions
- Code quality and Git history matter
- Measuring $>$ guessing

Break — 10 min

Theoretical Foundations

Theoretical Foundations
Why Can't We Just Add More Cores?

- **Intuition:** 8 cores should give 8× speedup, right?

Theoretical Foundations

Why Can't We Just Add More Cores?

- **Intuition:** 8 cores should give 8× speedup, right?
  - **Wrong!** Reality is more complex
- **Problems with perfect scaling:**
  1. **Not all code parallelizes** (sequential parts remain)
  2. **Overhead** (creating processes, communication, synchronization)
  3. **Load imbalance** (some cores finish before others)
  4. **Resource contention** (memory bandwidth, cache)
- **Amdahl's Law quantifies this:**
  - Theoretical maximum speedup
  - Based on sequential fraction
  - Pessimistic but realistic for fixed problem size
- **Gustafson's Law offers hope:**
  - Scale problem size with cores
  - "Weak scaling" vs. "strong scaling"
  - More optimistic for large-scale computing

Amdahl's Law

The Formula (Eijkhout 2.2.3.1)

General Speedup: $S = T_1/T_p$

Derivation: Break execution time into sequential and parallel parts

$$T_1 = T_{\text{seq}} + T_{\text{par}} = (1-P)\cdot T_1 + P \cdot T_1$$

$$T_p = T_{\text{seq}} + \frac{T_{\text{par}}}{p} = (1-P)\cdot T_1 + \frac{P \cdot T_1}{p}$$

Amdahl's Law:

$$S = \frac{T_1}{T_p} = \frac{1}{(1-P) + \frac{P}{p}}$$

Where: P = parallelizable fraction, p = number of cores

Upper limit: as $p \to \infty$, $\quad S \to S_{\max} = \dfrac{1}{1-P}$

$\Rightarrow$ The sequential fraction $(1-P)$ is a hard ceiling — more cores cannot help!

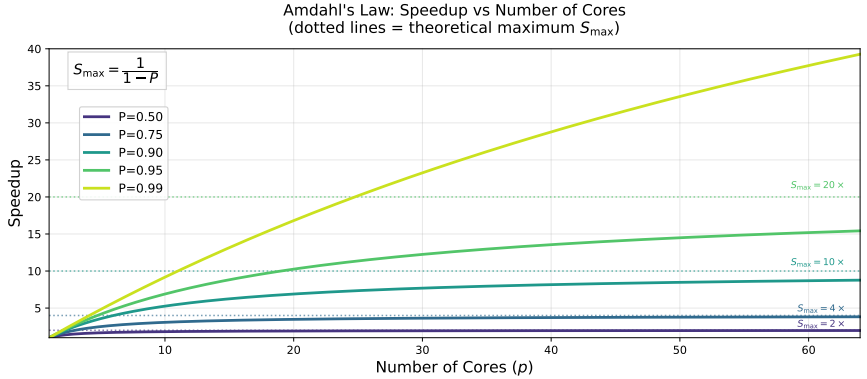
Amdahl's Law
Numerical Examples

P	p (cores)	Speedup S	$S_{\max} = \frac{1}{1-P}$
0.90	8	4.7×	10×
0.90	64	8.8×	10×
0.99	8	7.5×	100×
0.99	64	39×	100×

Reading the table:

- First two rows: 8-doubling p from 8→64 only adds 4.7 → 8.8× — diminishing returns
- Ceiling $S_{\max}$ stays at 10× regardless of p (determined by P alone)

Amdahl's Law
Visualization



Key takeaway:

- Getting close to S_{max} is difficult, for example, P=0.90 with $S_{max} = 10$ only reaches $S = 8$ by p=64.

Gustafson's Law
The Formula

Different assumption: Scale problem size with cores

Weak scaling: Keep time constant, increase workload

- Amdahl (strong): Same work, reduce time
- Gustafson (weak): Same time, increase work
- Example: 4 cores $\rightarrow$ 4$\times$ larger problem in same time

Derivation: If parallel work scales with p while serial work stays constant

$$\text{Work}_p = \underbrace{(1-P)\cdot \text{Work}_1}_{\text{serial (fixed)}} + P \cdot \text{Work}_1 \cdot p$$

$$\text{Speedup} = \frac{\text{Work}_p}{\text{Work}_1} = (1-P) + P \cdot p = p - (1-P)(p-1)$$

Key assumption: the serial part $(1-P)\cdot \text{Work}_1$ is *constant* — it does not grow with p.

Example: 90% parallel, 8 cores?

Gustafson's Law
The Formula

Different assumption: Scale problem size with cores

Weak scaling: Keep time constant, increase workload

- Amdahl (strong): Same work, reduce time
- Gustafson (weak): Same time, increase work
- Example: 4 cores $\rightarrow$ 4$\times$ larger problem in same time

Derivation: If parallel work scales with p while serial work stays constant

$$\text{Work}_p = \underbrace{(1-P)\cdot \text{Work}_1}_{\text{serial (fixed)}} + P \cdot \text{Work}_1 \cdot p$$

$$\text{Speedup} = \frac{\text{Work}_p}{\text{Work}_1} = (1-P) + P \cdot p = p - (1-P)(p-1)$$

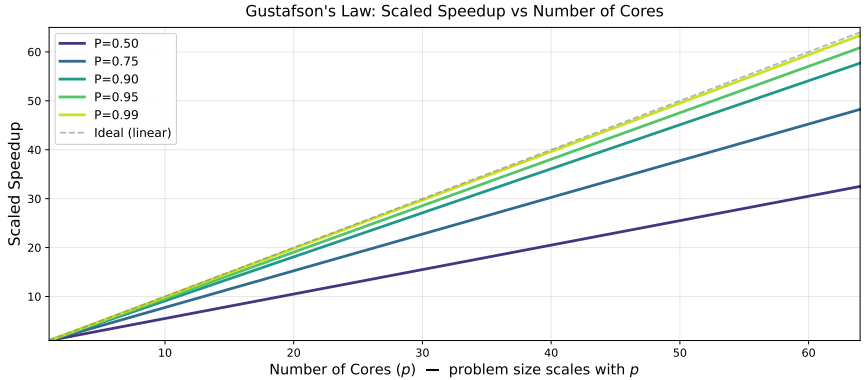
Key assumption: the serial part $(1-P)\cdot \text{Work}_1$ is *constant* — it does not grow with p.

Example: 90% parallel, 8 cores?

$$\text{Speedup} = 0.1 + 0.9 \times 8 = 7.3\times$$

Nearly linear scaling!

Gustafson's Law
Visualization



Key takeaways:

- Nearly linear scaling when P is high — no hard ceiling!
- Speedup $= (1-P) + P \cdot p$ grows without bound as p increases

Gustafson's Law
Comparison with Amdahl

Amdahl (Strong Scaling):

- Fixed problem size
- Example: Always 1024×1024
- Speedup limited by $(1 - P)$
- Pessimistic view

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{p}}$$

90% parallel, 8 cores: 4.7×

Gustafson (Weak Scaling):

- Scale problem with cores
- Example: 8 cores → 8192×8192
- Speedup scales with p
- Optimistic view

$$\text{Speedup} = (1 - P) + P \cdot p$$

90% parallel, 8 cores: 7.3×

This course: We test strong scaling (fixed grid size)

- You can render higher resolution if you want
- But we don't automatically scale resolution with cores

Work-Span Model

Work-Span Model

Beyond Amdahl's and Gustafson's Laws (Eijkhout 2.2.4)

Problem with Amdahl's and Gustafson's laws: Assume a fixed "parallelizable fraction" P. But *how parallel* is the parallel part?

Work-Span Model: Analyze parallelism from the algorithm's structure.

- **Work** W: Total number of operations (what a single processor does) [op]
- **Span** D: Longest chain of dependent operations (critical path) [op]
- **Parallelism** W/D: Maximum useful number of processors [dimensionless]

Intuition for W/D: Into how many independent slices can we cut W?

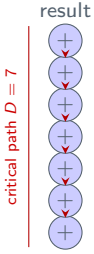
Key insight: Different algorithm structure w. different span D can have very different parallelism W/D for same W.

Example: Two ways to sum N numbers — see next slide.

Work-Span Example

Summing $N = 8$ numbers

Sequential sum $D = N - 1 = 7$

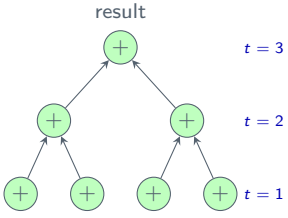


W=7, span D=7 (sequential chain)

$W/D = \mathbf{1}$ **only 1 processor useful**

For N=1024: $D = 1023 \Rightarrow W/D = \mathbf{1}$ (still!)

Tree reduction $D = \log_2 N = 3$



W=7, span D=3 (tree: $\log_2 N$ levels)

$W/D \approx \mathbf{2.3}$

For N=1024: $D = 10 \Rightarrow W/D \approx \mathbf{102}$!

Algorithm redesign can unlock parallelism even when W is unchanged.

Brent's Theorem
Scheduling Lower Bound

Brent's Theorem: Given W operations with critical path (span) D, the time on p processors is bounded by:

$$T_p \leq D + \frac{W - D}{p} \text{ [op]}$$

Interpretation: D = mandatory sequential cost you cannot avoid. $\frac{W-D}{p}$ = parallelizable remainder. $T_\infty = D$ (unlimited processors).

Example (cont.): Summing N numbers, $W = N - 1$ ops

		Tree reduction ($D = \log_2 N$)	
p	**Sequential** ($D = W = 7$)	N=8, D=3, W=7	N=1024, D=10, W=1023
2	$T_2 = \mathbf{7}$	$T_2 = 3 + \frac{4}{2} = \mathbf{5}$	$T_2 = 10 + \frac{1013}{2} \approx \mathbf{517}$
4	$T_4 = \mathbf{7}$	$T_4 = 3 + \frac{4}{4} = \mathbf{4}$	$T_4 = 10 + \frac{1013}{4} \approx \mathbf{264}$
∞	$T_\infty = \mathbf{7}$	$T_\infty = \mathbf{3}$	$T_\infty = \mathbf{10}$

Sequential: $D = W \Rightarrow T_p = W$ for all p — no parallelism. Tree: bound $\rightarrow D = \log_2 N$ as $p \rightarrow \infty$.

Practical use: If measured T_p greatly exceeds Brent's bound, you have overhead or load imbalance to investigate.

vs. Amdahl's Law: Brent is *more precise* — it works with task dependencies (D, W) rather than an assumed parallelizable fraction.

Work-Span Analysis
Mandelbrot Set

Let's analyze our Mandelbrot computation:

Step 1: What is the Work W?

- Grid: $N \times N$ pixels, each up to M iterations
- Per iteration: $\sim$8 FLOPs (complex multiply + add + escape test)
- $W \approx N^2 \cdot \bar{M} \cdot 8$ (where $\bar{M}$ is average iterations per pixel)
- For $N = 1024$, $\bar{M} \approx 50$: $W \approx 419$ million FLOPs

Step 2: What is the Span D?

- Pixels are *independent* — no pixel depends on another!
- $D = M_{\max} \cdot 8 =$ longest single-pixel chain $= 100 \times 8 = 800$ FLOPs

Step 3: What is the Parallelism P?

$$P = \frac{W}{D} = \frac{419 \times 10^6}{800} \approx 524{,}000$$

Mandelbrot is embarrassingly parallel! In principle, we could use half a million processors. On 8 cores, Brent predicts $T_8 \leq 800 + \frac{419 \times 10^6}{8} \approx \frac{W}{8}$ — nearly perfect speedup.

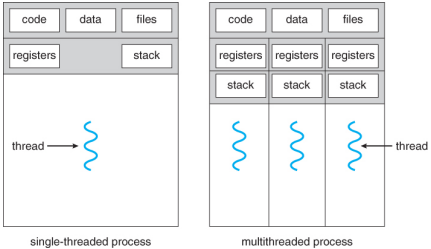
Studio exercise: Measure actual speedup. Why is it less than 8×?

Break — 10 min

Studio Session

Python Multiprocessing
Why Not Threading?

- **Python has two parallelism approaches:**
  1. **Threading:** Multiple threads in same process
  2. **Multiprocessing:** Multiple separate processes
- **The GIL Problem:** Global Interpreter Lock
  - GIL = mutex: only one thread runs Python code at a time
  - **Result:** Threading doesn't help for CPU-bound tasks!
- **When threading helps:**
  - I/O-bound tasks (waiting for network, disk)
  - GIL is released during I/O operations
- **Multiprocessing solution:**
  - Separate processes = separate Python interpreters
  - Each has its own GIL → true parallel execution
  - **Use this for Mandelbrot!**



Future: *Python 3.13 introduced an experimental free-threaded build (`python3.13t`) that removes the GIL — true thread parallelism without multiprocessing. Still opt-in and not yet production-ready.*

Python Multiprocessing
Basic Concepts

- **Process Pool:** Manager for worker processes

```
from multiprocessing import Pool

with Pool(processes=4) as pool:
    # pool manages 4 worker processes
    results = pool.map(function, data)
```

- **Key functions:**
  - **pool.map(func, iterable)**: Apply func to each item, return results
  - `pool.starmap(func, iterable)`: Like map, but unpacks arguments
  - `pool.imap(func, iterable)`: Iterator version (results yielded as computed)
  - `pool.apply_async(func, args)`: Single async call
- **Important constraints:**
  - Functions and data must be **picklable** — pickle is Python's built-in serialisation format; it converts objects to bytes so they can be sent to another process and reconstructed there
  - No lambdas, local functions, or complex objects (these cannot be serialised)
  - Data copied to each process (overhead!)
  - Results copied back (more overhead!)

Python Multiprocessing

How Many Processes? Logical vs. Physical Cores

Check physical count (`psutil`):

```
import psutil, os
print(os.cpu_count())                    # logical
print(psutil.cpu_count(logical=False)) # physical
```

- **Logical cores** = hardware threads visible to the OS
- **Hyperthreading** (Intel/AMD): one physical core presents as 2 logical*
- Beyond physical cores, extra workers *compete* rather than cooperate
- On Intel: 7 workers on 8-logical machine can beat 8 (OS gets a core)
- **Sweep** 1, 2, . . . , `cpu_count()` and plot speedup — find your optimum empirically

	Intel/AMD	**Apple Silicon**
Hyperthreading	Yes	No
Logical cores	2× physical	= physical
Core types	Uniform	P-cores + E-cores
Example	4-core i7 → 8	M1 Max: 8P+2E=10
Pitfall	HT inflates count	E-cores are slow

P = Performance core, E = Efficiency core

* *Two HT threads run simultaneously only if they use different execution units (e.g. one doing FP arithmetic while the other stalls on a cache miss); for Mandelbrot both compete for the same FPU — limited gain (if any) expected.*

Python Multiprocessing
Simple Example


```python
from multiprocessing import Pool
import time

def square(x):
    time.sleep(0.1) # Simulate work
    return x * x

if __name__ == '__main__':
    numbers = list(range(100))

    start = time.time()
    results_serial = [square(x) for x in numbers]
    time_serial = time.time() - start
    print(f"Serial: {time_serial:.2f}s")

    with Pool(processes=4) as pool:
        start_ = time.time()
        results_parallel = pool.map(square, numbers)

    time_parallel = time.time() - start_
    print(f"Parallel: {time_parallel:.2f}s")
    speedup = time_serial / time_parallel
    print(f"Speedup: {speedup:.2f}x")
```

Output:

```
$ python pool_example.py
Serial:   10.39s
Parallel:  3.00s
Speedup:   3.46x
```

100 tasks $\times$ 0.1 s each $=$ 10 s serial.
With 4 workers: 25 tasks each $=$ 2.5 s $+$ spawn overhead $\approx$ 3 s.

Speedup $< 4\times$ due to process spawn and IPC cost.

Python Multiprocessing
Common Pitfalls

- **Pitfall 1:** Forgetting `if __name__ == '__main__':`
  - Windows/Mac require this guard
  - Prevents infinite process spawning
  - Always use it for multiprocessing code!
- **Pitfall 2:** Trying to pickle un-picklable objects
  - Error: "Can't pickle lambda"
  - Solution: Use top-level functions, not lambdas
  - Solution: Use dill library for more complex cases
- **Pitfall 3:** Too fine-grained parallelism
  - Creating/destroying processes is expensive
  - Sending data between processes is expensive — *scales with return payload size*
  - Need enough work per task to overcome overhead
  - Rule of thumb: Each task should take >0.1s
- **Pitfall 4:** Not closing Pool
  - Processes may not terminate
  - Use `with Pool(...) as pool:` (context manager)
  - Or explicitly call `pool.close(); pool.join()`

Mini-Project 2
What is in MP2?

Due: Mon 7 Apr 23:59.

New implementations to add (on top of your MP1 Mandelbrot code):

- **Multiprocessing** — `Pool.map` with row-chunk parallelism (this session)
- **Dask** — local cluster (L05–L06)
- **Dask** — AAU Strato HPC cluster (L07)

What to hand in (same structure as MP1):

- Code: `.py` or `.ipynb` file(s)
- Performance notebook: timing table + speedup plot for all implementations
- Git log: `git log --format="%ad %h %s" --date=short --stat > git_log.txt`
- Reflection: 150+ words typed in the Moodle text field

⇒ **Start documenting your Mandelbrot parallel results today — your performance notebook entries count toward MP2.**

Studio Session

Overview — today's plan

Quick walkthrough, then you work independently.

Exercises (practice — no hand-in):

E1 — MC π Serial Write and time `estimate_pi_serial()`

E2 — MC π Parallel Parallelize with `Pool.map()`; sweep 1... `cpu_count()` workers

E3 — MC π Analysis Speedup curve; efficiency table; back-solve implied serial fraction

MP2 Milestones (document results in performance notebook):

M1 — Parallel Mandelbrot Refactor to chunk-based Numba kernels

M2 — Parallel Mandelbrot Implement `Pool.map` wrapper; verify result

M3 — Parallel Mandelbrot Benchmark sweep; speedup and efficiency table

Soft wrap-up $\sim$**15:30** — share one result even if still working

- After 15:30: keep working, ask for help — session runs until 16:15
- Commit your work before you leave!

Monte Carlo π

Monte Carlo π
The Algorithm

Concept:

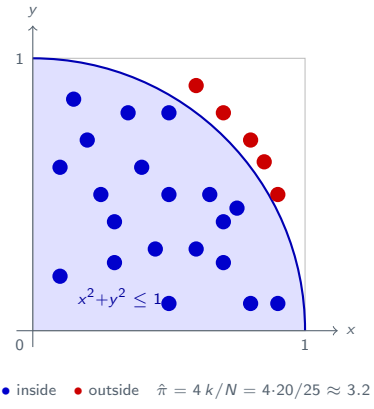
- Random points in unit square $[0,1]\times[0,1]$
- Count points inside quarter circle
- Ratio $\approx \pi/4$
- More points $\rightarrow$ better estimate

Formula:

$$\pi \approx 4 \cdot \frac{\text{points in circle}}{\text{total points}}$$

Why good for parallelization?

- Independent random samples $\rightarrow$ "Embarrassingly parallel"
- Minimal IPC: each worker receives one integer (sample count), returns one integer (hit count) — no large data transfer



Monte Carlo π

Exercise 1: Serial Implementation (15 min)

Write `estimate_pi_serial(num_samples)` — a plain Python loop:

1. Draw two uniform random numbers $x, y \in [0, 1)$
2. Count a *hit* when $x^2 + y^2 \leq 1$
3. Return $4 \times$ hits/num_samples

Time it: use `time.perf_counter()` with 3 runs, take `statistics.median()`. Try `num_samples = 10_000_000`.

Questions:

- How accurate is the estimate? Run several times — does it vary?
- What is the serial time? This will be your speedup denominator in E3.

✓ **Done?** Note your serial time → commit

Monte Carlo π

Exercise 1: Code example


```python
import math, random, time, statistics

def estimate_pi_serial(num_samples):
    inside_circle = 0
    for _ in range(num_samples):
        x, y = random.random(), random.random()
        if x*x + y*y <= 1:
            inside_circle += 1
    return 4 * inside_circle / num_samples

if __name__ == '__main__':
    num_samples = 10_000_000
    times = []
    for _ in range(3):
        t0 = time.perf_counter()
        pi_estimate = estimate_pi_serial(num_samples)
        times.append(time.perf_counter() - t0)
    t_serial = statistics.median(times)
    print(f"pi estimate: {pi_estimate:.6f}  (error: {abs(pi_estimate-math.pi):.6f})")
    print(f"Serial time: {t_serial:.3f}s")
```

Monte Carlo π

Exercise 2: Parallel Implementation (20 min)

Parallelize using `Pool.map` — each worker returns a hit count (an integer):

- Worker function `estimate_pi_chunk(num_samples)`: runs the loop, returns `inside_circle`
- Main process: sum worker results, divide by total samples to get $\hat{\pi}$
- Why good IPC? Input: one integer per task. Output: one integer per task. No arrays.

Sweep `num_processes` from 1 to `os.cpu_count()`. For each worker count: measure elapsed time and compute speedup.

Questions:

- Do all worker counts give the same $\hat{\pi}$? Why or why not?
- At which count do you first see a meaningful speedup?

✓ **Done?** Check $\hat{\pi}$ is consistent across runs → continue to E3

Monte Carlo π

Exercise 2: Code example

```python
from multiprocessing import Pool
import os, random, time, statistics

def estimate_pi_chunk(num_samples):
    inside_circle = 0
    for _ in range(num_samples):
        x, y = random.random(), random.random()
        if x*x + y*y <= 1:
            inside_circle += 1
    return inside_circle

def estimate_pi_parallel(num_samples, num_processes=4):
    samples_per_process = num_samples // num_processes
    tasks = [samples_per_process] * num_processes
    with Pool(processes=num_processes) as pool:
        results = pool.map(estimate_pi_chunk, tasks)
    return 4 * sum(results) / num_samples

if __name__ == '__main__':
    num_samples = 10_000_000
    for num_proc in range(1, os.cpu_count() + 1):
        times = []
        for _ in range(3):
            t0 = time.perf_counter()
            pi_est = estimate_pi_parallel(num_samples, num_proc)
            times.append(time.perf_counter() - t0)
        t = statistics.median(times)
        print(f"{num_proc:2d} workers: {t:.3f}s  pi={pi_est:.6f}")
```

Monte Carlo π

Exercise 3: Analyze Results (25 min)

Parallel efficiency: $E_p = S_p/p$ — fraction of ideal speedup per core (100% = perfect).

For each worker count (1 to `cpu_count()`), tabulate: workers | time (s) | speedup S_p | efficiency E_p (%)

Questions to discuss:

1. At which worker count p^* is speedup *maximum*?
2. Does speedup plateau or *drop* beyond p^*? Why?
3. **Back-solve implied serial fraction:** $s = \dfrac{1/S_{p^*} - 1/p^*}{1 - 1/p^*}$ — what fraction of time is effectively serial (IPC overhead + spawning)?
4. **Mac M1/M2/M3 users:** do you see a slope change near worker 8 (E-cores)?

(Optional) Plot: workers (x) vs. speedup (y) — actual and ideal linear.

✓ **Done?** Discuss with a neighbour → move on to MP2 milestones

Parallel Mandelbrot

Parallel Mandelbrot
Strategy

- **Key insight:** Each pixel is independent — "embarrassingly parallel"
- **Approach: split the $N \times N$ grid into horizontal row chunks**
  - Divide N rows into `n_workers` equal-sized chunks
  - Each process receives only scalar parameters (row range + bounds) — minimal IPC input
  - Each process computes all pixels in its rows independently
  - Each process returns a 2D `int32` array of iteration counts
  - Main process assembles result with `np.vstack()`
- **Load balancing note:**
  - Some rows converge faster (points outside the set escape early)
  - Row-based split may be slightly uneven — acceptable as a first approach

Parallel Mandelbrot — MP2 M1

Refactor Code (20 min)

Refactor your MP1 Numba code into three functions (two `@njit`, one plain wrapper):

`mandelbrot_pixel(c_real, c_imag, max_iter)` The scalar kernel you already have from L03. Returns the escape iteration count for a single complex point.

`mandelbrot_chunk(row_start, row_end, N, x_min, x_max, y_min, y_max, max_iter)` Loops over rows `[row_start, row_end)` and all columns. Computes pixel coordinates from index + bounds — no arrays received as input. Returns a `(row_end - row_start)` × `N` `int32` array.

`mandelbrot_serial(N, x_min, ...)` Thin wrapper: calls `mandelbrot_chunk(0, N, ...)` — the whole grid as one chunk.

✓ **Done?** Serial result matches your L03 output → commit

Parallel Mandelbrot — MP2 M1

Code example


```python
# mandelbrot_parallel.py  (Tasks 1-3 are one continuous script)
import numpy as np
from numba import njit
from multiprocessing import Pool
import time, os, statistics, matplotlib.pyplot as plt
from pathlib import Path

@njit
def mandelbrot_pixel(c_real, c_imag, max_iter):
    z_real = z_imag = 0.0
    for i in range(max_iter):
        zr2 = z_real*z_real
        zi2 = z_imag*z_imag
        if zr2 + zi2 > 4.0: return i
        z_imag = 2.0*z_real*z_imag + c_imag
        z_real = zr2 - zi2 + c_real
    return max_iter

@njit
def mandelbrot_chunk(row_start, row_end, N,
                     x_min, x_max, y_min, y_max, max_iter):
    out = np.empty((row_end - row_start, N), dtype=np.int32)
    dx = (x_max - x_min) / N
    dy = (y_max - y_min) / N
    for r in range(row_end - row_start):
        c_imag = y_min + (r + row_start) * dy
        for col in range(N):
            out[r, col] = mandelbrot_pixel(x_min + col*dx, c_imag, max_iter)
    return out

def mandelbrot_serial(N, x_min, x_max, y_min, y_max, max_iter=100):
    return mandelbrot_chunk(0, N, N, x_min, x_max, y_min, y_max, max_iter)
```

Parallel Mandelbrot — MP2 M2

Parallel Implementation (25 min)

- **Write** `mandelbrot_parallel(N, x_min, x_max, y_min, y_max, max_iter, n_workers)`
  - Build a list of chunk tuples: `(row_start, row_end, N, x_min, x_max, y_min, y_max, max_iter)`
  - Use `pool.map(_worker, chunks)` to distribute across workers
  - Reassemble with `np.vstack(parts)`
- **Pass only parameters — not a pre-computed grid**
  - In MP1 we pre-created the full complex grid `C` and passed it to the iteration algorithm — passing a slice of that array to each worker would mean sending ∼8 KB per task over IPC
  - Instead: pass only scalar bounds `(x_min, x_max, y_min, y_max, N)` and let each worker compute its own coordinates
  - IPC input per task: ∼60 bytes (8 scalars) vs. ∼8 KB (1024-element `float64` array)
  - IPC output per task is unavoidable: `chunk_size` × `N` × 4 bytes (the result array)
- **Wrapper function** `_worker(args)`
  - `pool.map` calls `func(item)` — it takes only one argument
  - `mandelbrot_chunk` takes 8 arguments, so `_worker` receives the packed tuple and unpacks it
  - Normally, this can be done with a lambda function, but lambdas are not picklable, so it must be defined as function at module level.

✓ **Done?** Verify result matches serial → commit

Parallel Mandelbrot — MP2 M2

Code example


```python
# --- MP2 M2: add below M1 in mandelbrot_parallel.py ---

def _worker(args):
    return mandelbrot_chunk(*args)

def mandelbrot_parallel(N, x_min, x_max, y_min, y_max,
                        max_iter=100, n_workers=4):
    chunk_size = max(1, N // n_workers)
    chunks, row = [], 0
    while row < N:
        row_end = min(row + chunk_size, N)
        chunks.append((row, row_end, N, x_min, x_max, y_min, y_max, max_iter))
        row = row_end

    with Pool(processes=n_workers) as pool:
        pool.map(_worker, chunks)       # un-timed warm-up: Numba JIT in workers
        parts = pool.map(_worker, chunks)

    return np.vstack(parts)

if __name__ == '__main__':
    result = mandelbrot_parallel(1024, -2.5, 1.0, -1.25, 1.25, n_workers=4)

    fig, ax = plt.subplots(figsize=(8, 6))
    ax.imshow(result, extent=[-2.5, 1.0, -1.25, 1.25],
              cmap='inferno', origin='lower', aspect='equal')
    ax.set_xlabel('Re(c)')
    ax.set_ylabel('Im(c)')
    out = Path(__file__).parent / 'mandelbrot.png'
    fig.savefig(out, dpi=150)
    print(f'Saved: {out}')
```

Parallel Mandelbrot — MP2 M3

Benchmark (15 min)

- **Sweep** `n_workers` **from 1 to** `os.cpu_count()`; for each worker count:
  1. Create a fresh `Pool(processes=n_workers)`
  2. Run one **un-timed** `pool.map(_worker, chunks)` — this triggers Numba JIT compilation in every worker process; without this, JIT time is included in your measurement
  3. Run 3 **timed** `pool.map` calls; take `statistics.median()`
  4. Record: time, speedup $S_p = t_{\text{serial}}/t_p$, efficiency $E_p = S_p/p$
- **Serial baseline:** call `mandelbrot_serial` 3 times after your M1 warm-up (Numba already compiled in main process); take median
- **Where to place the timer:**
  - Start `t0` immediately before `pool.map()`
  - Stop immediately after `np.vstack(parts)` — include assembly time
  - Keep `Pool` creation *outside* the timed region

Parallel Mandelbrot — MP2 M3

Code example


```python
# --- MP2 M3: benchmark (in __main__ block) ---
N, max_iter = 1024, 100
X_MIN, X_MAX, Y_MIN, Y_MAX = -2.5, 1.0, -1.25, 1.25

# Serial baseline (Numba already warm after M1 warm-up)
times = []
for _ in range(3):
    t0 = time.perf_counter()
    mandelbrot_serial(N, X_MIN, X_MAX, Y_MIN, Y_MAX, max_iter)
    times.append(time.perf_counter() - t0)
t_serial = statistics.median(times)

for n_workers in range(1, os.cpu_count() + 1):
    chunk_size = max(1, N // n_workers)
    chunks, row = [], 0
    while row < N:
        end = min(row + chunk_size, N)
        chunks.append((row, end, N, X_MIN, X_MAX, Y_MIN, Y_MAX, max_iter))
        row = end
    with Pool(processes=n_workers) as pool:
        pool.map(_worker, chunks)          # warm-up: Numba JIT in all workers
        times = []
        for _ in range(3):
            t0 = time.perf_counter()
            np.vstack(pool.map(_worker, chunks))
            times.append(time.perf_counter() - t0)
    t_par = statistics.median(times)
    speedup = t_serial / t_par
    print(f"{n_workers:2d} workers: {t_par:.3f}s, speedup={speedup:.2f}x,  eff={speedup/n_workers*100:.0f}%")
```

✓ **Done?** Record speedup table in performance notebook (MP2) → commit

Studio Wrap-up (∼15:30)

Share one result — then keep working

Quick round: who reached what?

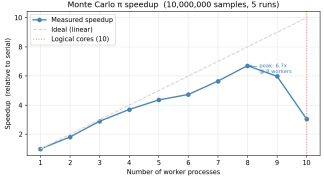
- **E1/E2**: MC π parallel — at which worker count was speedup maximum?
- **E3**: What is your implied serial fraction s? Does it match the IPC payload pattern?
- **MP2 M1/M2**: Mandelbrot refactored and parallel result verified?
- **MP2 M3**: Benchmark complete — what speedup did you achieve?

After 15:30:

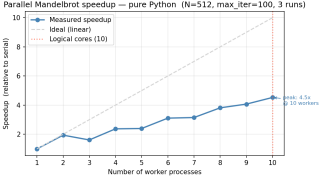
- Keep working on remaining milestones — studio runs until 16:15
- Ask for help; Jimmy circulates
- Commit your work before you leave!

Results: Speedup Curves

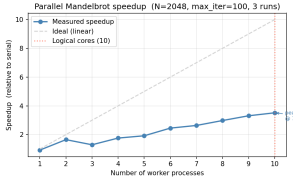
M1 Max (8 P-cores + 2 E-cores)



MC π (1 int/worker)



Naive Mand. (∼100 KB/worker)



Numba Mand. (∼1.6 MB/worker)

- **IPC payload is the key variable** — same parallelism, very different speedup
- MC π: near-ideal to p=8 P-cores; E-cores hurt (compute bottleneck at p=9–10)
- Naive Mand.: E-cores *help* — chunks are heavy enough that slower cores still contribute
- Numba Mand.: IPC (1.6 MB/worker) dominates — even 10 workers give only 3.5×

Connecting to Amdahl's Law

Back-solving the implied serial fraction (Eijkhout 2.2.3.1)

If Amdahl's Law holds: $S_{\text{peak}} = \dfrac{1}{s + (1-s)/p}$. Rearranging for implied s:

$$s = \frac{1/S_{\text{peak}} - 1/p_{\text{peak}}}{1 - 1/p_{\text{peak}}}$$

	MC π	**Naive Mand. ($N = 512$)**	**Numba Mand. ($N = 2048$)**
Peak speedup	6.71× at p=8	4.53× at p=10	3.52× at p=10
Implied s	$\approx 2.7\%$	$\approx 13\%$	$\approx 20\%$
IPC payload	1 int	∼100 KB	∼1.6 MB

- MC π: $s \approx 2.7\%$ — almost nothing is serial; the Brent bound (8×) is nearly reached
- Numba Mand.: $s \approx 20\%$ even though the algorithm is fully parallel — pickling a 1.6 MB array *per worker* is the hidden serial bottleneck (Eijkhout 2.2.3.1)
- **Key lesson:** implied s scales with IPC payload, not algorithmic serial fraction

Connecting to Gustafson's Law

Scaled speedup: how big a problem can p workers solve in the same time?

Gustafson's Law (Eijkhout 2.2.3.2): fix *time*, scale the *problem*.

$$S_{\mathrm{G}} = s + (1 - s) \cdot p$$

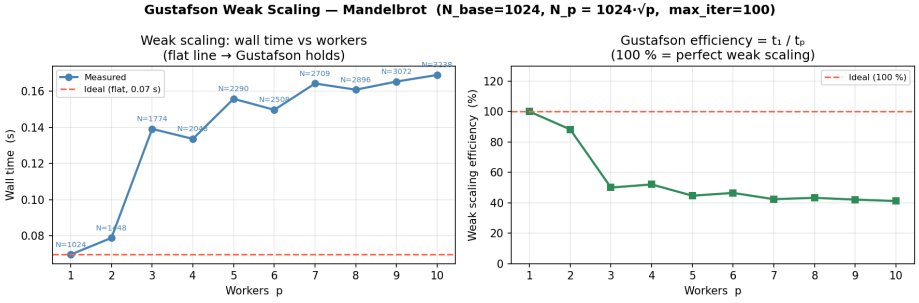
Using the same implied s from the Amdahl back-solve:

	MC π	Naive Mand.	Numba Mand.
Implied s	2.7%	13%	20%
S_G at peak p	7.81× (p=8)	8.83× (p=10)	8.2× (p=10)
Actual speedup	6.71×	4.53×	3.52×

- $S_G \gg S_{\text{actual}}$: Gustafson scales the problem to fill cores; the serial fraction is diluted
- For Mandelbrot (s=20%, p=10): $S_G \approx 8.2\times$ means rendering an 8.2× larger image in the same wall time — $N = 2048 \to 2048 \times \sqrt{8.2} \approx 5870$ pixels wide
- **Amdahl** = pessimistic (fixed workload); **Gustafson** = optimistic (fixed time) — same s, different questions

Gustafson: Weak Scaling Experiment

Scale $N_p = N_{\text{base}} \cdot \sqrt{p}$ so each worker always computes N_{base}^2 pixels



- **Expected:** flat wall time — each worker always does the same amount of work
- **Actual:** sharp drop at $p = 3$ (50% efficiency), plateau $\sim$41% for $p \geq 3$
- **Why?** IPC payload per worker is also $\propto N_{\text{base}}^2$ — *both* compute and transfer scale identically. Increasing N_{base} doesn't help: the ratio stays fixed. The "hidden serial fraction" is pickling overhead — solved by shared memory (L06: Dask)

Wrap-up

Wrap-up
What We Accomplished Today

Theory:

- Amdahl's Law, Gustafson's Law, back-solving implied serial fraction
- Work-Span model, Brent's theorem, parallel efficiency

Python multiprocessing:

- Process pools, `Pool.map()`, chunksize, IPC overhead
- GIL and why multiprocessing beats threading for CPU-bound work

Studio:

- Monte Carlo π parallelised and analysed (speedup curve, implied s)
- Parallel Mandelbrot started — row-chunk approach, Numba + multiprocessing

Before Next Week

Complete at Home

Must complete (if not done today):

1. Finish Parallel Mandelbrot MP2 milestones (M2 + M3 benchmark)
2. Don't forget to commit work

Prepare for Week 5: Parallel Computing 2 (Thu 12 Mar)

- Listen to the Week 5 podcast or watch the video
- Read textbook sections

Next week (Week 5):

- Granularity — fine vs. coarse, chunk size effects
- Graham's bound — makespan and scheduling limits
- Map-filter-reduce — functional parallel patterns
- MP2 — Parallel Mandelbrot analysis continues

Questions?

Contact:
Jimmy Jessen Nielsen
jjn@es.aau.dk